

Learning Accurate Integer Transformer Machine-Translation Models

Ephrem Wu

Xilinx, Inc., San Jose, USA

ephrem.wu@xilinx.com

Abstract

We describe a method for training accurate Transformer machine-translation models to run inference using 8-bit integer (INT8) hardware matrix multipliers, as opposed to the more costly single-precision floating-point (FP32) hardware. Unlike previous work, which converted only 85 Transformer matrix multiplications to INT8, leaving 48 out of 133 of them in FP32 because of unacceptable accuracy loss, we convert them *all* to INT8 without compromising accuracy. Tested on the *newstest2014* English-to-German translation task, our INT8 Transformer Base and Transformer Big models yield BLEU scores that are 99.3% to 100% relative to those of the corresponding FP32 models. Our approach converts all matrix-multiplication tensors from an existing FP32 model into INT8 tensors by automatically making range-precision trade-offs during training. To demonstrate the robustness of this approach, we also include results from INT6 Transformer models.

1 Introduction

We report a method for training accurate yet compact Transformer machine-translation models [Vaswani *et al.*, 2017]. Specifically, we aim these models at hardware with 8-bit integer (INT8) matrix multipliers. Compared to single-precision floating-point (FP32) matrix multiplications, INT8 matrix multiplications not only reduce both storage and bandwidth four times, but they also consume 15 times less energy [Horowitz, 2014]. We therefore have two goals: 1) to convert *all* matrix multiplications from FP32 to INT8, and 2) to maintain translation accuracy relative to the FP32 model.

The Transformer model has proven to be a powerful attention-based model for machine translation [Bahdanau *et al.*, 2015], and has inspired much follow-up research in this area. For example, the term “Transformer” appears 56 times in the findings of the 2018 Conference of Machine Translation (WMT18) [Bojar *et al.*, 2018b] and 105 times in WMT19 [Barrault *et al.*, 2019]. The Transformer model in [Vaswani *et al.*, 2017] comes in two forms, Transformer Base, which has 61 million parameters, and Transformer Big, which has 210 million parameters. These model sizes are large compared

to the convolutional neural network benchmark ResNet-50 [He *et al.*, 2016], a 25M-parameter model, but still small compared to OpenAI GPT-2 models [Radford *et al.*, 2019], a family of Transformer models with 117M, 345M, 774M, and 1.6B parameters. Transformer model sizes show no signs of reduction. For instance, [Huang *et al.*, 2019] reports multi-lingual machine-translation results even for larger Transformer models for up to 6 billion parameters. It is therefore useful to explore techniques for reducing parameter representation costs, for instance, going from FP32 to INT8, rather than holding out for smaller Transformer architectures.

2 Related Work

We draw inspiration from three papers on quantizing machine-translation models to INT8. The first paper, [Wu *et al.*, 2016], was published before the Transformer [Vaswani *et al.*, 2017]. The authors of this paper quantized parameter (weight) tensors but not non-parameter tensors in a range-preserving fashion. The second paper, [Juncys-Dowmunt *et al.*, 2018], reported INT8 Transformer Big English-to-German translation results but did not report any results for INT8 Transformer Base. The third paper, [Bhandare *et al.*, 2019], did the opposite: it reported INT8 Transformer Base English-to-German translation results but not for Transformer Big.

Before the Transformer paper was published, [Wu *et al.*, 2016] described a quantization-aware training method for an attention-based LSTM neural machine translator. They treated parameter tensors differently from non-parameter tensors. In particular, they quantized parameter tensors to INT8 in a range-preserving fashion. For non-parameters, they treated logits differently from other non-parameter tensors. Specifically, they clipped logits to $[-25.0, 25.0]$, and they clipped the rest of the non-parameter tensors to $[-8.0, 8.0]$ and annealed them to $[-1.0, 1.0]$ by the end of the training. In the LSTM module, matrix multiplication operands were 8-bit integers, and accumulators were 16-bit integers. All other operations in the LSTM module were 16-bit operations. The softmax function and the attention mechanism remained as floating-point operations. These authors reported an English-to-German translation BLEU score of 24.61 (*newstest2014*). Similarly, we quantized parameters to 8-bit integers in a range-preserving manner. We attempted to clip floating-point non-parameter tensors before uniform quanti-